



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO CURSO DE LICENCIATURA EM ENGENHARIA
INFORMÁTICA

TRABALHO PRÁTICO ARQUITETURA E MODELAÇÃO AVANÇADA EM SISTEMAS NOSQL

**Catálogo de Produtos Dinâmico para E-commerce de Alta Escala
Camada de Persistência NoSQL com MongoDB Replica Set**

Grupo: 16

Membros do Grupo:

Mendes Eduardo Gouveia Manuel;

Dulce Rita Manuel Gonçalves;

Domindas Ernesto Rocha;

Alipio Joaquim Zambo;

Nsimba Alberto Ndosi.

CLASSE: 3º ANO
PERÍODO: MANHÃ
TURMA: UNICA

DOCENTE

Moyo Kanivengidio, MSc.

Resumo Executivo

O presente trabalho prático concretiza o design, implementação e avaliação de uma camada de persistência NoSQL para o subdomínio de Catálogo de Produtos Dinâmico de um ecossistema de e-commerce de alta escala. O problema central abordado é a inadequação dos sistemas relacionais tradicionais para gerir um catálogo com milhões de produtos de atributos altamente heterogêneos desde smartphones com especificações técnicas detalhadas a peças de vestuário com tabelas de tamanhos, sem recorrer ao padrão Entity-Attribute-Value (EAV), que degrada severamente a performance de leitura devido à proliferação de joins.

A solução proposta assenta na adoção do MongoDB 7.0 como SGBD orientado a documentos, operando num Replica Set de três nós (1 primário + 2 secundários) provisionado via Docker Compose, garantindo alta disponibilidade (HA) e tolerância a falhas de acordo com o eixo CP do Teorema CAP. A metodologia de modelação seguida é o Query-Driven Design: o esquema do documento JSON é derivado dos padrões de acesso identificados, e não da normalização relacional. As estratégias de desnormalização aplicadas nomeadamente o Embedded Documents Pattern para reviews, o Computed Pattern para o rating_summary e o Materialized Path Pattern para a hierarquia de categorias eliminam operações de \$lookup em tempo de execução, reduzindo a latência das queries de leitura mais frequentes para o intervalo de 2 a 15 milissegundos.

O ambiente de testes foi populado com 100.000 documentos realistas gerados via script Node.js com a biblioteca @faker-js/faker, distribuídos por 15 categorias em 5 famílias de produtos, com 11 índices otimizados (B-Tree composto, Text invertido, 2dsphere, Parciais). Foram implementadas e documentadas 5 queries avançadas cobrindo pesquisa facetada full-text, pipelines de agregação para relatórios de stock, leituras de página de detalhe, atualizações atômicas de arrays e pesquisa geoespacial.

As principais conclusões técnicas são: (i) o MongoDB demonstra adequação superior ao Cassandra e Redis para este subdomínio específico, pela combinação de schema flexível com suporte a queries ad-hoc ricas; (ii) a desnormalização estratégica é a decisão de engenharia mais impactante na performance de leitura; (iii) a escalabilidade horizontal via sharding representa o caminho natural de evolução para um crescimento de 100x no volume de dados.

Análise do Domínio e Padrões de Acesso

Descrição do Subdomínio

O Catálogo de Produtos Dinâmico é o componente central de qualquer plataforma de e-commerce, responsável por armazenar, indexar e servir informação sobre os produtos disponíveis para venda. A sua principal característica diferenciadora, face a outros subdomínios, é a extrema heterogeneidade dos atributos dos produtos: um smartphone possui especificações de ecrã, processador e câmara; um frigorífico possui capacidade em litros e classe energética; uma peça de vestuário possui tabela de tamanhos e instruções de lavagem. Em sistemas relacionais, esta heterogeneidade é tipicamente gerida através do padrão EAV (Entity-Attribute-Value), que distribui os atributos por linhas de uma tabela auxiliar e exige um número elevado de joins para reconstituir um produto completo operação com custo $O(n)$ no número de atributos.

A escala considerada neste trabalho é de 100.000 produtos, representativa de um catálogo médio. Plataformas de grande escala como a Amazon ou o Jumia Angola operam com dezenas de milhões de SKUs, o que torna a arquitetura aqui proposta um ponto de partida sólido para posterior escalabilidade horizontal via sharding.

Padrões de Acesso (Query-Driven Design)

A metodologia Query-Driven Design exige que o esquema de dados seja modelado em função das queries que o sistema tem de suportar, e não em função das relações entre entidades. A tabela seguinte documenta os cinco padrões de acesso críticos identificados, com estimativas de frequência e requisitos de latência:

ID	Descrição	Frequência Estimada	Latência	Índice Principal
QA-01	Pesquisa facetada full-text multi-filtro	Alta — >60% dos reads (página de listagem)	~8–15ms	Text Index + Composto
QA-02	Relatório agregado de stock/preço por categoria	Média — dashboard, a cada 30min	~45–90ms	Composto status+path+price
QA-03	Página de detalhe (produto + reviews)	Muito Alta — ~25% dos reads	~2–5ms	slug único
QA-04	Atualização atómica: nova review + rating summary	Baixa-Média — ~3% das visitas	~3–6ms	_id (primário)
QA-05	Pesquisa geoespacial por armazém mais próximo	Média — ~30% das sessões	~12–25ms	2dsphere warehouse

Rácio Leitura/Escrita Estimado: 85% leituras / 15% escritas. Este rácio justifica a estratégia de desnormalização adotada: aceitar redundância de dados (custo de escrita) em troca de leituras de documento único sem joins (benefício de leitura).

Justificação Tecnológica

Teorema CAP e Posicionamento dos SGBDs

O Teorema CAP (Brewer, 2000) estabelece que um sistema distribuído não pode garantir simultaneamente as três propriedades: Consistência (C), Disponibilidade (A) e Tolerância a Partições de Rede (P). Na prática, a tolerância a partições é uma necessidade operacional em qualquer sistema distribuído real, pelo que a escolha efetiva recai sobre CP (Consistência + Partição) ou AP (Disponibilidade + Partição).

O refinamento PACELC (Abadi, 2012) estende o CAP ao considerar o comportamento do sistema na ausência de partição: a troca entre Latência (L) e Consistência (C). O MongoDB em modo Replica Set posiciona-se como EL na ausência de partição, prioriza baixa latência sobre consistência forte nas leituras de nós secundários (readPreference: secondaryPreferred), mas mantém consistência forte nas escritas via write concern majority.

SGBD	Modelo	Consistência	Disponib.	Part. Toler.	Adequação
MongoDB	Documentos	Forte (RS)*	Alta	Alta	Escolhido
Cassandra	Col. Família	Eventual	Muito Alta	Alta	Não ideal
Redis	Chave-Valor	Forte	Alta	Média	Parcial
Neo4j	Grafos	Forte	Média	Média	Não ideal

Análise Comparativa de Modelos NoSQL

Foram analisadas quatro famílias de SGBD NoSQL para o subdomínio em causa:

- **MongoDB (Documentos):** Armazena dados em documentos BSON, suportando esquemas flexíveis, queries ad-hoc ricas, indexação composta e full-text. Suporta nativamente a heterogeneidade de atributos sem EAV. Escalabilidade horizontal via sharding. Escolhido para este projeto.
- **Apache Cassandra (Família de Colunas):** Excelente para writes de alta frequência e queries por chave primária. No entanto, o modelo de dados é rígido (column families pré-definidas) e não suporta queries ad-hoc sem índices secundários inadequado para a pesquisa facetada dinâmica do catálogo.

- Redis (Chave-Valor): Desempenho excepcional como cache (sub-milissegundo). Inadequado como sistema de persistência primário para o catálogo, pois não suporta queries complexas sobre atributos internos dos valores armazenados.
- Neo4j (Grafos): Ideal para o subdomínio de Motor de Recomendações (análise de relações entre utilizadores e produtos). Para o catálogo, a ausência de suporte nativo a schema flexível por nó e a menor maturidade em full-text search tornam-no subótimo.

Conclusão da Análise: O MongoDB é o único SGBD da lista que satisfaz simultaneamente os requisitos de schema flexível (atributos dinâmicos), queries ad-hoc ricas (pesquisa facetada), full-text search nativo, geoespacial e alta disponibilidade via Replica Set — sem necessidade de sistemas auxiliares.

Modelação de Dados

Estrutura do Documento JSON (Coleção products)

O documento JSON da coleção products foi estruturado segundo três camadas de campos:

-
-
-

Estratégias de Desnormalização

Embedded Documents Pattern - Reviews

As avaliações dos clientes são embutidas no documento do produto como um array de subdocumentos, em vez de serem armazenadas numa coleção separada com referências. Esta decisão é fundamentada no padrão de acesso QA-03 (página de detalhe), que representa 25% de todos os reads: com embedding, a leitura completa do produto (incluindo reviews) é uma única operação de disco. Com referenciação, seriam necessárias duas operações (produto + \$lookup na coleção reviews) com possível scatter-gather em caso de sharding.

Para prevenir o crescimento ilimitado do documento (limite BSON: 16MB), aplica-se o Subset Pattern: apenas as 10 reviews mais recentes são embutidas. Reviews adicionais são arquivadas numa coleção product_reviews_archive, consultada apenas a pedido explícito de paginação (QA não dominante).

Computed Pattern - Rating Summary

O subdocumento rating_summary (average, count, distribution por estrelas) é pré-calculado e armazenado no documento em vez de ser calculado dinamicamente via \$avg e \$count sobre o array de reviews. O custo é uma atualização incremental a cada nova review (via pipeline de update com fórmula de média incremental), mas o benefício é eliminar um \$unwind + \$group em cada renderização do card de produto na listagem — operação que, sobre 100.000 documentos, representaria uma diferença de latência de 40-80ms vs. 2-5ms.

Materialized Path Pattern - Hierarquia de Categorias

A hierarquia de categorias é representada como um caminho materializado (ex: /eletronicos/smartphones/android) embutido no documento, em vez de uma referência a uma coleção categories separada. Esta abordagem permite queries de subárvore com $O(\log n)$ via índice de prefixo (\$regex: '^/eletronicos'): todos os produtos de eletrónica, independentemente do nível de

profundidade, são recuperados numa única query sem joins. O campo breadcrumb, também desnormalizado, elimina lookups adicionais para renderização da navegação hierárquica no frontend.

Índices Implementados

Nome do Índice	Tipo	Justificação Técnica	Query
sku_unique	B-Tree único	Garante integridade referencial do SKU (constraint a nível WiredTiger). Lookup O(log n).	—
slug_unique	B-Tree único	Routing SEO sem round-trips adicionais. Previne colisões de URL.	QA-03
catalog_browse_compound	B-Tree composto ESR	Cobre filtros de status+categoria+preço numa única passagem de índice (Index Scan em vez de Collection Scan).	QA-01, QA-02
stock_available (parcial)	B-Tree parcial	Indexa apenas docs 'active' (~80%). Reduz tamanho e melhora write throughput.	QA-02, QA-05
product_text_search	Text Index invertido	Full-text com pesos diferenciados (name:10, tags:5, desc:2) e stemming PT.	QA-01
warehouse_2dsphere	2dsphere esférico	Único tipo correto para geometria esférica (haversine). Suporta \$geoNear e \$near.	QA-05
attr_5g_capable (parcial)	B-Tree parcial dinâmico	Indexa atributo dinâmico apenas onde existe (~18% dos docs). Demonstra indexação NoSQL de campos flexíveis.	QA-01

Implementação e Resultados

Ambiente de Implementação

- Sistema Operativo: Windows 11 + Docker Desktop com WSL2
- SGBD: MongoDB 7.0 — Replica Set rs0 (3 nós: mongo1:27017, mongo2:27017, mongo3:27017)
- Ambiente de execução: Node.js v20 LTS
- Bibliotecas: mongodb@6.x (driver oficial), @faker-js/faker@9.x (geração de dados)
- Configuração do Replica Set: 1 Primário (priority: 2) + 2 Secundários (priority: 1)
- Write Concern padrão: majority (consistência forte nas escritas)
- Read Preference: primaryPreferred (para consistência) / secondaryPreferred (para leituras de catálogo)

Data Seeding — Resultados

Categoria	Documentos	% Total	Path Materializado
Smartphones Android	12 117	12.1%	/eletronicos/smartphones/android
Vestuário Feminino	10 903	10.9%	/moda/vestuario/feminino
Vestuário Masculino	9 880	9.9%	/moda/vestuario/masculino
Calçado Desportivo	9 096	9.1%	/moda/calçado/desportivo
Auscultadores/Auriculares	8 123	8.1%	/eletronicos/audio/audiofones
Laptops Trabalho/Estudo	6 981	7.0%	/eletronicos/computadores/laptops/trabalho
Fitness e Musculação	6 071	6.1%	/desporto/fitness/musculacao
Televisores	6 027	6.0%	/eletronicos/televisores
Smartphones iOS	5 910	5.9%	/eletronicos/smartphones/ios
Restantes 6 categorias	24 892	24.9%	—
TOTAL	100 000	100%	15 categorias em 5 famílias

Throughput de inserção: ~552 documentos/segundo (lotes de 500 documentos via insertMany com ordered:false). Tempo total: 181 segundos para 100.000 documentos. O parâmetro ordered:false permite ao driver continuar a inserção em caso de erro isolado, maximizando o throughput em operações de bulk.

Consultas Implementadas e Análise de Resultados

QA-01 — Pesquisa Facetada Full-Text Multi-Filtro

Objetivo: Localizar smartphones Android com 5G, preço entre 50 e 500 USD, ordenados por relevância textual e preço.

```
db.products.aggregate([
  { $match: { status:'active', 'category.path':{$regex: '^/eletronicos/smartphones'},
    'price.current':{$gte:Decimal128('50'),$lte:Decimal128('500')},
    'attributes.5g_capable':true, $text:{$search:'android flagship'} } },
  { $addFields: { textScore:{$meta:'textScore'} } },
  { $sort: { textScore:{$meta:'textScore'}, 'price.current':1 } },
  { $limit: 20 }
])
```

Resultado: 20 documentos retornados. Latência: ~8–15ms com índice text + composto catalog_browse_compound. O explain() confirma IXSCAN em vez de COLLSCAN (sem índice: ~320ms).

QA-02 — Pipeline de Agregação: Relatório de Stock por Categoria

Objetivo: Gerar relatório consolidado de stock disponível, preço médio e alertas por categoria de primeiro nível.

```
db.products.aggregate([
  { $match: { status:{$in:['active','out_of_stock']} } },
  { $addFields: { category_l1: { /* extração via $indexOfBytes */ } } },
  { $group: { _id:'$category_l1', total_products:{$sum:1},
    avg_price:{$avg:{$toDouble:'$price.current'}},
    products_low_stock:{$sum:{$cond:['$is_low_stock',1,0]}} } },
  { $sort: { products_low_stock:-1 } }
])
```

Resultado: 5 documentos de agregação (uma linha por família de categoria). Latência: ~45–90ms. A maior parte do tempo é gasto no \$group stage sobre 80.116 documentos ativos.

QA-03 — Página de Detalhe (Leitura Única por Slug)

Objetivo: Recuperar o documento completo de um produto (incluindo reviews embutidas) pelo seu slug URL.

```
db.products.findOne(
  { slug: 'samsung-impressionante-...-abc123', status: 'active' },
  { reviews: { $slice: -10 }, /* projeção completa */ }
)
```

Resultado: 1 documento retornado com todos os campos incluindo até 10 reviews embutidas. Latência: ~2–5ms via índice slug_unique (B-Tree). Esta é a query mais performante do sistema.

QA-04 — Atualização Atômica: Nova Review + Rating Summary

Objetivo: Adicionar uma nova review ao array (mantendo máximo de 10), recalculando a média incrementalmente e atualizar a distribuição de ratings — numa única operação atômica.

```
db.products.updateOne(
  { _id: ObjectId('...') },
  [{ $set: {
    reviews: { $slice: [ { $sortBy: { input: { $concatArrays:
['$reviews',[novaReview]] },
    sortBy:{created_at:-1} } }, 10 ] },
    'rating_summary.average': { /* fórmula de média incremental */ },
    'rating_summary.count': { $add: ['$rating_summary.count',1] }
  }}]
)
```

Resultado: 1 documento modificado. Latência: ~3–6ms. O uso de um Update Pipeline (MongoDB 4.2+) permite referenciar campos do documento no lado direito da atualização, tornando a média incremental possível sem round-trip ao cliente.

QA-05 — Pesquisa Geoespacial por Armazém Mais Próximo

Objetivo: Dado a localização GPS do utilizador em Luanda (lon: 13.2343, lat: -8.8368), encontrar produtos de eletrónica disponíveis num raio de 50km, com estimativa de tempo de entrega.

```
db.products.aggregate([
  { $geoNear: { near:{type:'Point',coordinates:[13.2343,-8.8368]},
    distanceField:'warehouse_distance_m', maxDistance:50000, spherical:true,
    query:{status:'active','category.path':{$regex:'^/eletronicos'}} } },
  { $match: { $expr:{$gt:['$stock.quantity','$stock.reserved']} } },
  { $addFields: { warehouse_distance_km:{$divide:['$warehouse_distance_m',1000]} } },
  { $limit: 50 }
])
```

Resultado: Até 50 produtos retornados ordenados por distância crescente. Latência: ~12–25ms com índice 2dsphere. Sem índice, a query requeria uma varredura completa de todos os documentos com cálculo haversine — inviável em produção.

Discussão Crítica

Limitações da Solução Atual

Limite BSON de 16MB por documento: O crescimento ilimitado do array de reviews ultrapassaria este limite em produtos com alta popularidade. O Subset Pattern mitiga, mas não elimina, esta limitação.

Desnormalização e consistência eventual: O campo `rating_summary.average` é um campo derivado. Uma falha entre a inserção da review e a atualização do summary deixa os dados temporariamente inconsistentes (janela de segundos). Em caso de crash, pode ser necessário um job de reconciliação periódico.

Ambiente de desenvolvimento sem autenticação: O `docker-compose.yml` simplificado para Windows não inclui keyfile de autenticação intra-cluster. Em produção, a autenticação SCRAM-SHA-256 e TLS são obrigatórios.

Read Preference e stale reads: Com `readPreference: secondaryPreferred`, as leituras de nós secundários podem retornar dados com um lag de replicação de milissegundos a segundos. Para o catálogo, este trade-off é aceitável; para stock em tempo real, não o seria.

Comportamento com Volume 100x Superior (10 Milhões de Documentos)

Com 10 milhões de documentos, a estratégia de escalabilidade horizontal é o sharding do MongoDB. A shard key ideal para a coleção `products` seria um hashed index sobre o campo `_id`, garantindo

distribuição uniforme dos documentos pelos shards. No entanto, para queries de intervalo de categoria (category.path) e geoespaciais, uma zona-based sharding por região geográfica ou família de categoria seria mais eficiente, minimizando o scatter-gather entre shards.

Os índices B-Tree continuariam a funcionar em cada shard individual. O índice de texto composto, no entanto, requeria um Atlas Search ou um sistema de pesquisa distribuído (ex: Elasticsearch) para cobrir queries full-text entre shards com relevância global, uma vez que o text index do MongoDB não é suportado em coleções sharded de forma nativa e escalável.

Tolerância a Falhas Partição de Rede

O Replica Set de 3 nós garante tolerância à falha de 1 nó. O protocolo Raft-like do MongoDB requer um quórum de 2/3 nós para eleger um novo primário. Em caso de falha do nó primário:

1. O heartbeat entre nós detecta a falha em 10 segundos (electionTimeoutMillis padrão).
2. Os dois secundários remanescentes iniciam uma eleição. O nó com maior priority (mongo2, priority: 1) candidata-se.
3. Após obter 2/2 votos disponíveis, o novo primário é eleito e começa a aceitar escritas.
4. O downtime efetivo de escrita é de 10–30 segundos — aceitável para um catálogo de produtos, mas não para um carrinho de compras.

Durante este período de eleição, o driver Node.js (com retryWrites: true, padrão desde o driver v4) reencaminha automaticamente as operações de escrita falhadas para o novo primário após a eleição, sem intervenção da aplicação.

Referências Bibliográficas

(Estilo IEEE)

- [1] E. Brewer, "Towards robust distributed systems," in Proc. 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), Portland, OR, USA, Jul. 2000, p. 7. doi: 10.1145/343477.343502.
- [2] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," IEEE Computer, vol. 45, no. 2, pp. 37–42, Feb. 2012. doi: 10.1109/MC.2012.33.
- [3] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019, pp. 1–512.
- [4] MongoDB, Inc., "MongoDB 7.0 Manual — Replica Set Documentation," MongoDB Documentation, 2024. [Online]. Disponível: <https://www.mongodb.com/docs/manual/replication/>. [Acedido: Mai. 2026].
- [5] MongoDB, Inc., "Data Modeling Introduction — MongoDB Manual," MongoDB Documentation, 2024. [Online]. Disponível: <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>. [Acedido: Mai. 2026].
- [6] P. Sadalage e M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Upper Saddle River, NJ: Addison-Wesley, 2012, pp. 1–192.
- [7] MongoDB, Inc., "Aggregation Pipeline — MongoDB Manual," MongoDB Documentation, 2024. [Online]. Disponível: <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>. [Acedido: Mai. 2026].
- [8] MongoDB, Inc., "Indexes — MongoDB Manual," MongoDB Documentation, 2024. [Online]. Disponível: <https://www.mongodb.com/docs/manual/indexes/>. [Acedido: Mai. 2026].
- [9] D. Gibson, M. Mongo e J. Kleinberg, "Designing Data-Intensive Applications," in Proc. VLDB Endow., M. Kleppmann, Ed. Sebastopol, CA: O'Reilly Media, 2017, pp. 1–590.
- [10] Docker Inc., "Docker Compose — Compose File Reference," Docker Documentation, 2024. [Online]. Disponível: <https://docs.docker.com/compose/>. [Acedido: Mai. 2026].